

Question Code: Q332

Open the Learnlytica assessment tool

Enter your Email address, and then enter the Question ID and click on start assessment

Once started, click on VSCode, and it will directly open the `solution.py` file of the question, and write your code there.

Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.

After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

Problem Statement

You need to implement a **Hashtag Analyzer** that takes social media captions as input and extracts useful hashtag information.

The implementation should handle:

Keys → Hashtags

Values → Hashtag frequencies

The implementation should handle case insensitivity and remove duplicate formatting issues.

Class Declaration

class HashtagAnalyzer:

 No need for init method

 If used, follow recommendations -

```
def __init__(self):  
    pass
```

Operations

1. Extract and Normalize Hashtags

 Extracts hashtags from a caption, converts them to lowercase, and removes punctuation attached at the end.

- ◆ **Function Prototype:**

```
def extract_hashtags(self, caption: str) -> list:
```

- ◆ **Example Input:**

```
extract_hashtags("Loving the weather! #Travel #Food, #travel")
```

- ◆ **Expected Output:**

```
['travel', 'food', 'travel']
```

- ◆ **Implementation Flow:**

-  Convert the input caption to lowercase.

-  Split the caption into words using `.split()`.

-  Identify words that start with `#`.

-  Remove `#` from the beginning of the hashtag.

💡 Remove punctuation using `re.sub(r' [^\w\s]', ' ', word)`.

💡 Return the list of cleaned hashtags.

2. Compute Hashtag Frequency

📌 Counts occurrences of each hashtag in a dictionary.

♦ Function Prototype:

```
def compute_hashtag_frequency(self, hashtags: list) -> dict:
```

♦ Example Input:

```
compute_hashtag_frequency(['travel', 'food', 'travel'])
```

♦ Expected Output:

```
{'travel': 2, 'food': 1}
```

♦ Implementation Flow:

💡 Declare an empty dictionary.

💡 Iterate through the hashtag list using for loop.

💡 Use `dictionary.get(hashtag, 0)` to count occurrences.

💡 Return a dictionary where keys = hashtags and values = frequencies.

3. Get Most Used Hashtag

📌 Returns the most frequently used hashtag and its count.

- ◆ **Function Prototype:**

```
def get_most_used_hashtag(self, freq_dict: dict) -> tuple:
```

- ◆ **Example Input:**

```
get_most_used_hashtag({'travel': 2, 'food': 1})
```

- ◆ **Expected Output:**

```
('travel', 2)
```

- ◆ **Implementation Flow:**

💡 If nothing is present in dictionary, return **None**.

💡 Iterate through the dictionary to find the hashtag with the highest count.

💡 Use **max()** for finding highest frequency.

💡 Use **lambda x: x[1]** for mapping values.

💡 Return the hashtag and its count as a tuple.

4. Filter Hashtags by Minimum Frequency

📌 Returns hashtags that appear at least **n** times.

- ◆ **Function Prototype:**

```
def filter_hashtags_by_frequency(self, freq_dict: dict, n: int) -> dict:
```

- ◆ **Example Input:**

```
filter_hashtags_by_frequency({'travel': 2, 'food': 1}, 2)
```

♦ **Expected Output:**

```
{'travel': 2}
```

♦ **Implementation Flow:**



Iterate over the dictionary using for loop.



Include only hashtags whose count is greater than or equal to **n**.



Return the filtered dictionary.



If no hashtag satisfies the condition, return an empty dictionary.

Q332 Driver Code: HashtagAnalyzer

```
import re
```

```
# Trainee will write this class in solution.py
```

```
# class HashtagAnalyzer:
```

```
#     def extract_hashtags(self, caption: str) -> list:
```

```
#         pass
```

```
#
```

```
#     def compute_hashtag_frequency(self, hashtags: list) ->  
dict:
```

```
#         pass
```

```
#  
  
# def get_most_used_hashtag(self, freq_dict: dict) ->  
tuple:  
  
#     pass  
  
#  
  
# def filter_hashtags_by_frequency(self, freq_dict: dict, n:  
int) -> dict:  
  
#     pass
```

```
if __name__ == "__main__":  
    analyzer = HashtagAnalyzer()
```

```
    caption = "Loving the weather! #Travel #Food, #travel  
#Nature! #food"
```

```
    hashtags = analyzer.extract_hashtags(caption)  
    print("Extracted Hashtags:", hashtags)
```

```
    freq = analyzer.compute_hashtag_frequency(hashtags)  
    print("Hashtag Frequency:", freq)
```

```
most_used = analyzer.get_most_used_hashtag(freq)
```

```
print("Most Used Hashtag:", most_used)
```

```
filtered = analyzer.filter_hashtags_by_frequency(freq, 2)
```

```
print("Filtered Hashtags:", filtered)
```

Question Code: Q333

Open the Learnlytica assessment tool

Enter your Email address, and then enter the Question ID and click on start assessment

Once started, click on VSCode, and it will directly open the `solution.py` file of the question, and write your code there.

Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.

After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

Problem Statement

You need to implement a **Product Review Rating Analyzer** that takes a list of customer ratings and performs simple analysis.

The implementation should:

Keys → Rating values

Values → Rating frequencies

The implementation should ignore invalid ratings and analyze only ratings between 1 and 5.

Class Declaration

```
class ProductReviewAnalyzer:
```

 No need for init method

 If used, follow recommendations -

```
def __init__(self):
```

```
    pass
```

Operations

1. Clean and Validate Ratings

 Filters only valid ratings from the input list.

◆ **Function Prototype:**

```
def clean_ratings(self, ratings: list) -> list:
```

◆ **Example Input:**

```
clean_ratings([5, 4, 3, 5, 6, 0, 4, "bad"])
```

◆ **Expected Output:**

```
[5, 4, 3, 5, 4]
```


♦ Implementation Flow:



Declare an empty list.



Iterate through the input ratings list using for loop.



Check whether each rating is an integer.



Include only ratings greater than or equal to 1 and less than or equal to 5.



Return the cleaned ratings list.

2. Compute Rating Frequency



Counts occurrences of each rating in a dictionary.

♦ Function Prototype:

```
def compute_rating_frequency(self, ratings: list) -> dict:
```

♦ Example Input:

```
compute_rating_frequency([5, 4, 3, 5, 4])
```

♦ Expected Output:

```
{5: 2, 4: 2, 3: 1}
```

♦ Implementation Flow:



Declare an empty dictionary.



Iterate through the list using for loop.



Use `dictionary.get(rating, 0)` to count occurrences.

💡 Return a dictionary where keys = ratings and values = frequencies.

3. Get Most Common Rating

📌 Returns the most common rating and its count.

- ◆ **Function Prototype:**

```
def get_most_common_rating(self, freq_dict: dict) -> tuple:
```

- ◆ **Example Input:**

```
get_most_common_rating({5: 2, 4: 2, 3: 1})
```

- ◆ **Expected Output:**

```
(5, 2)
```

- ◆ **Implementation Flow:**

💡 If nothing is present in dictionary, return **None**.

💡 Iterate through the dictionary to find the rating with the highest count.

💡 Use **max()** for finding highest frequency.

💡 Use **lambda x: x[1]** for mapping values.

💡 Return the rating and its count as a tuple.

4. Filter Ratings by Minimum Frequency

📌 Returns ratings that appear at least **n** times.

- ◆ **Function Prototype:**

```
def filter_ratings_by_frequency(self, freq_dict: dict, n: int) -> dict:
```

- ◆ **Example Input:**

```
filter_ratings_by_frequency({5: 2, 4: 2, 3: 1}, 2)
```

- ◆ **Expected Output:**

```
{5: 2, 4: 2}
```

- ◆ **Implementation Flow:**



Iterate over the dictionary using for loop.



Include only ratings whose count is greater than or equal to **n**.



Return the filtered dictionary.



If no rating satisfies the condition, return an empty dictionary.

Q333 Driver Code: ProductReviewAnalyzer

```
# Trainee will write this class in solution.py
```

```
# class ProductReviewAnalyzer:
```

```
#     def clean_ratings(self, ratings: list) -> list:
```

```
#         pass
```

```
#
```

```
#     def compute_rating_frequency(self, ratings: list) -> dict:
```

```
#     pass

#

#     def get_most_common_rating(self, freq_dict: dict) ->
tuple:

#     pass

#

#     def filter_ratings_by_frequency(self, freq_dict: dict, n:
int) -> dict:

#     pass
```

```
if __name__ == "__main__":
    analyzer = ProductReviewAnalyzer()

    ratings = [5, 4, 3, 5, 6, 0, 4, "bad", 2, 5, 1, 4]

    clean = analyzer.clean_ratings(ratings)
    print("Clean Ratings:", clean)

    freq = analyzer.compute_rating_frequency(clean)
    print("Rating Frequency:", freq)
```

```
most_common = analyzer.get_most_common_rating(freq)
print("Most Common Rating:", most_common)
```

```
filtered = analyzer.filter_ratings_by_frequency(freq, 2)
print("Filtered Ratings:", filtered)
```

Question Code: Q334

Open the Learnlytica assessment tool

Enter your Email address, and then enter the Question ID and click on start assessment

Once started, click on VSCode, and it will directly open the `solution.py` file of the question, and write your code there.

Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.

After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

Problem Statement

You need to implement an Email Domain Counter that takes a list of email addresses as input and returns a dictionary where:

Keys → Email domains

Values → Domain frequencies

The implementation should handle case insensitivity and ignore invalid email addresses.

Class Declaration

class EmailDomainCounter:

 **No need for init method**

 **If used, follow recommendations -**

```
def __init__(self):
```

```
    pass
```

Operations

1. Extract and Normalize Domains

 **Extracts domain names from valid email addresses and converts them to lowercase.**

◆ **Function Prototype:**

```
def extract_domains(self, emails: list) -> list:
```

- ◆ **Example Input:**

```
extract_domains(["Amit@Gmail.com",  
"riya@yahoo.com", "invalid-email",  
"john@Gmail.com"])
```

- ◆ **Expected Output:**

```
['gmail.com', 'yahoo.com', 'gmail.com']
```

- ◆ **Implementation Flow:**



Declare an empty list.



Iterate through the email list using for loop.



Convert each email to lowercase.



Check whether the email contains "@".



Split the email using "@".



If domain part is not empty, append it to the result list.



Return the domain list.

2. Compute Domain Frequency



Counts occurrences of each domain in a dictionary.

- ◆ **Function Prototype:**

```
def compute_domain_frequency(self, domains: list) ->  
dict:
```

- ◆ Example Input:

```
compute_domain_frequency(['gmail.com', 'yahoo.com', 'gmail.com'])
```

- ◆ Expected Output:

```
{'gmail.com': 2, 'yahoo.com': 1}
```

- ◆ Implementation Flow:



Declare an empty dictionary.



Iterate through the domain list using for loop.



Use `dictionary.get(domain, 0)` to count occurrences.



Return a dictionary where keys = domains and values = frequencies.

3. Get Most Common Domain



Returns the most common email domain and its count.

- ◆ Function Prototype:

```
def get_most_common_domain(self, freq_dict: dict) -> tuple:
```

- ◆ Example Input:


```
get_most_common_domain({'gmail.com': 2,  
'yahoo.com': 1})
```

◆ Expected Output:

```
('gmail.com', 2)
```

◆ Implementation Flow:

💡 If nothing is present in dictionary, return **None**.

💡 Iterate through the dictionary to find the domain with the highest count.

💡 Use **max()** for finding highest frequency.

💡 Use **lambda x: x[1]** for mapping values.

💡 Return the domain and its count as a tuple.

4. Filter Domains by Minimum Frequency

📌 Returns domains that appear at least **n** times.

◆ Function Prototype:

```
def filter_domains_by_frequency(self, freq_dict: dict, n:  
int) -> dict:
```

◆ Example Input:

```
filter_domains_by_frequency({'gmail.com': 2,  
'yahoo.com': 1}, 2)
```

◆ Expected Output:

{'gmail.com': 2}

◆ **Implementation Flow:**



Iterate over the dictionary using for loop.



Include only domains whose count is greater than or equal to **n.**



Return the filtered dictionary.



If no domain satisfies the condition, return an empty dictionary.

Q334 Driver Code: EmailDomainCounter

Trainee will write this class in solution.py

class EmailDomainCounter:

def extract_domains(self, emails: list) -> list:

pass

#

def compute_domain_frequency(self, domains: list) -> dict:

pass

```
#  
  
# def get_most_common_domain(self, freq_dict: dict) ->  
tuple:  
  
#     pass  
  
#  
  
# def filter_domains_by_frequency(self, freq_dict: dict, n:  
int) -> dict:  
  
#     pass
```

```
if __name__ == "__main__":  
    counter = EmailDomainCounter()
```

```
emails = [  
    "Amit@Gmail.com",  
    "riya@yahoo.com",  
    "invalid-email",  
    "john@Gmail.com",  
    "meera@outlook.com",  
    "sam@yahoo.com",  
    "wrong@"
```

```
"test@GMAIL.com"  
]
```

```
domains = counter.extract_domains(emails)  
print("Extracted Domains:", domains)
```

```
freq = counter.compute_domain_frequency(domains)  
print("Domain Frequency:", freq)
```

```
most_common =  
counter.get_most_common_domain(freq)  
print("Most Common Domain:", most_common)
```

```
filtered = counter.filter_domains_by_frequency(freq, 2)  
print("Filtered Domains:", filtered)
```

Question Code: Q335

Open the Learnlytica assessment tool

Enter your Email address, and then enter the Question ID and click on start assessment

Once started, click on VSCode, and it will directly open the **solution.py** file of the question, and write your code there.

Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.

After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

Problem Statement

You need to implement a **File Extension Counter** that takes a list of file names as input and returns a dictionary where:

Keys → File extensions

Values → Extension frequencies

The implementation should handle case insensitivity and ignore invalid file names without extensions.

Class Declaration

```
class FileExtensionCounter:
```

 No need for init method

 If used, follow recommendations -

```
def __init__(self):
```

```
    pass
```

Operations

1. Extract and Normalize Extensions

 Extracts file extensions from valid file names and converts them to lowercase.

- ◆ **Function Prototype:**

```
def extract_extensions(self, files: list) -> list:
```

- ◆ **Example Input:**

```
extract_extensions(["report.PDF", "image.png", "notes",  
"data.CSV", "photo.PNG"])
```

- ◆ **Expected Output:**

```
['pdf', 'png', 'csv', 'png']
```

- ◆ **Implementation Flow:**

-  Declare an empty list.

-  Iterate through the file list using for loop.

-  Convert each file name to lowercase.

-  Check whether the file name contains ".".

-  Split the file name using ".".

-  If extension part is not empty, append it to the result list.

-  Return the extension list.

2. Compute Extension Frequency

 Counts occurrences of each extension in a dictionary.

- ◆ **Function Prototype:**

```
def compute_extension_frequency(self, extensions: list) -> dict:
```

- ◆ **Example Input:**

```
compute_extension_frequency(['pdf', 'png', 'csv', 'png'])
```

- ◆ **Expected Output:**

```
{'pdf': 1, 'png': 2, 'csv': 1}
```

- ◆ **Implementation Flow:**



Declare an empty dictionary.



Iterate through the extension list using for loop.



Use `dictionary.get(extension, 0)` to count occurrences.



Return a dictionary where keys = extensions and values = frequencies.

3. Get Most Common Extension



Returns the most common file extension and its count.

- ◆ **Function Prototype:**

```
def get_most_common_extension(self, freq_dict: dict) -> tuple:
```

- ◆ **Example Input:**

```
get_most_common_extension({'pdf': 1, 'png': 2, 'csv': 1})
```

- ◆ **Expected Output:**

```
('png', 2)
```

- ◆ **Implementation Flow:**



If nothing is present in dictionary, return `None`.

💡 Iterate through the dictionary to find the extension with the highest count.

💡 Use `max()` for finding highest frequency.

💡 Use `lambda x: x[1]` for mapping values.

💡 Return the extension and its count as a tuple.

4. Filter Extensions by Minimum Frequency

📌 Returns extensions that appear at least `n` times.

◆ **Function Prototype:**

```
def filter_extensions_by_frequency(self, freq_dict: dict, n: int) -> dict:
```

◆ **Example Input:**

```
filter_extensions_by_frequency({'pdf': 1, 'png': 2, 'csv': 1}, 2)
```

◆ **Expected Output:**

```
{'png': 2}
```

◆ **Implementation Flow:**

💡 Iterate over the dictionary using for loop.

💡 Include only extensions whose count is greater than or equal to `n`.

💡 Return the filtered dictionary.

💡 If no extension satisfies the condition, return an empty dictionary.

Q335 Driver Code: FileExtensionCounter

Trainee will write this class in solution.py

class FileExtensionCounter:

def extract_extensions(self, files: list) -> list:

pass

#

def compute_extension_frequency(self, extensions:
list) -> dict:

pass

#

def get_most_common_extension(self, freq_dict: dict)
-> tuple:

pass

#

def filter_extensions_by_frequency(self, freq_dict: dict,
n: int) -> dict:

pass

```
if __name__ == "__main__":  
    counter = FileExtensionCounter()
```

```
    files = [  
        "report.PDF",  
        "image.png",  
        "notes",  
        "data.CSV",  
        "photo.PNG",  
        "summary.pdf",  
        "archive.zip",  
        "README",  
        "table.csv"  
    ]
```

```
    extensions = counter.extract_extensions(files)  
    print("Extracted Extensions:", extensions)
```

```
    freq = counter.compute_extension_frequency(extensions)  
    print("Extension Frequency:", freq)
```

```
most_common =  
counter.get_most_common_extension(freq)  
  
print("Most Common Extension:", most_common)  
  
filtered = counter.filter_extensions_by_frequency(freq, 2)  
print("Filtered Extensions:", filtered)
```